

Take-Home: Transaction Ledger API

Overview

Build an HTTP API for a simple money-movement ledger. Accounts hold balances; clients can transfer funds between accounts and query balances and transaction history.

This is a small problem to state, but a deep one to get right. We care far more about **how you reason** than about how much you ship. A clean, correct core beats a sprawling, half-finished system every time.

You have **one week** from the day you receive this. After you submit, we'll schedule an interview to walk through your solution together, your design decisions, your tradeoffs, and what you'd do differently. Treat the interview as the main event, the code is what we talk *about*.

What to build

An HTTP API with at least these operations:

- **Create an account.** An account has a balance. You decide how it's created and what the starting state is.
- **Transfer funds** from one account to another.
- **Get an account's balance.**
- **List an account's transactions.**

You choose the language, framework, and datastore. Use whatever you'd reach for to build something real.

Hard requirements

These are the point of the exercise. Everything else is secondary.

1. **Transfers must be idempotent.** A client that retries the same transfer must not move money twice. A retry should be safe and should return a sensible result.
2. **Money is conserved.** Balances must never go negative, and money must never be created or destroyed. The sum of what leaves one account must equal what arrives in another.
3. **Correctness holds under concurrency.** The two requirements above must survive concurrent requests, including multiple requests touching the same account at the same instant.

4. **It persists to a real datastore** and runs locally with clear setup instructions. We should be able to clone, follow your README, and have it running quickly.

Operating assumption: multiple instances

Assume this service runs as **multiple stateless instances behind a load balancer**, all backed by a single shared datastore. Any request can land on any instance, and you have no control over which one.

You do **not** need to actually deploy multiple instances or set up any infrastructure. We want your **code and design** to be correct under this model. In particular, be ready to discuss how your guarantees hold when concurrent requests for the same account are handled by different instances, an original request and its retry, for example, need not land on the same process.

Scope and time

We'd expect a strong candidate to spend roughly **3–5 focused hours** on this. The week is there to fit around your schedule and life.

Build the core cleanly, then **tell us what you deliberately left out and why**. Knowing what *not* to build is part of what we're evaluating. Things you might reasonably scope out (and mention in your README) include authentication, pagination, rate limiting, currency conversion, audit logging, and so on. Leaving these out is fine — leaving them out *on purpose, with a reason* is what we're looking for.

Using AI tools

Using AI assistants (Claude, Copilot, ChatGPT, etc.) is completely fine and expected. But **own every decision in your submission**.

What to submit

- A repository (or zip) with your code.
- A **README** that covers:
 - How to set up and run it locally.
 - The key design decisions you made, and why.
 - How you satisfy each of the three hard requirements — idempotency, conservation of money, and correctness under concurrency across multiple instances.
 - What you chose not to build, and what you'd add next given more time.
- A few tests demonstrating the behavior that matters most. You don't need exhaustive coverage; show us the tests *you'd* prioritize and be ready to say why.

What we're evaluating

- **Correctness of the core invariants**, especially under concurrency and retries.
- **Sound judgment about tradeoffs** — and the ability to articulate them.
- **Clarity** — of the code, the API, and the README.
- **Scope discipline** — solving the problem asked, not the problem imagined.
- **Ownership** — you understand and can defend every part of what you submitted.

Good luck. We're looking forward to the conversation.